Jackett Torznab Sidecar — Deployment & Validation Guide

> Status: deployment-ready (compose fragment + `.env.example` keys + validation script authored; no container booted in this authoring pass). Companion: `jackett-local-stack-research.md` (decision dossier) and `jackett-local-stack-implementation.md` (Go Torznab client). Operator decision implemented: **Jackett as a Torznab sidecar in the Lava local stack, fronted by `lava-api-go`, mDNS-discovered, NOT bundled in the APK.**

This guide covers how to start the sidecar, validate it end-to-end with one command, perform the one-time (stateful, partly manual) Jackett indexer setup for IPTorrents (with FlareSolverr), and how the Android app reaches it.

---

1. Topology

```
Android app ──mDNS(_lava-api._tcp)──▶ lava-api-go (:8443, host-net, TLS+HMAC)
                                              │  (server-side HTTP, holds api_key)
                                              ▼
                                  127.0.0.1:9117  →  lava-jackett (NOT advertised)
                                              │
                                  (only when profile=cloudflare)
                                              ▼
                                  127.0.0.1:8191  →  lava-flaresolverr
```

- The app discovers **exactly one** service type — `_lava-api._tcp` (= `lava-api-go`, default `LAVA_API_MDNS_TYPE` in `lava-api-go/internal/discovery/mdns.go`). Jackett has no mDNS advertisement and is **never** put on the LAN.
- `lava-api-go` runs `network_mode: host` (mDNS requires host net). A host-net container **cannot** resolve the bridge DNS name `lava-jackett`, so the sidecar's port is published to the **host loopback** (`127.0.0.1:9117`), and `lava-api-go` reaches it at `http://`

127.0.0.1:9117. Loopback bind keeps it off the LAN (same posture as lava-postgres's 127.0.0.1:8432).
- **FlareSolverr** (headless Chromium) is only needed for Cloudflare-protected trackers (notably **IPTorrents**). It is heavy, so it lives behind the cloudflare profile and is OFF by default. Jackett reaches it over the lava-net bridge at http://lava-flaresolverr:8191 (both are bridge members); the loopback publish of 8191 is only for the validation curl.

---

2. Files

File	Role
tools/lava-containers/docker-compose.jackett.yml	compose fragment — lava-jackett (+ optional lava-flaresolverr) on lava-net, loopback-published
scripts/validate-jackett-sidecar.sh	one-command end-to-end readiness check (bring up → wait healthy → probe caps + flaresolverr → teardown)
.env.example	placeholders for all LAVA_API_JACKETT_* + LAVA_JACKETT_* + LAVA_FLARESOLVERR_* keys
lava-api-go/internal/jackett/	the Go Torznab client lava-api-go uses (see implementation doc)
lava-api-go/internal/config/config.go	reads LAVA_API_JACKETT_* env (the consumer side)

---

3. Environment variables (§6.R — nothing hardcoded)

Copy .env.example → .env and fill these (the .env is gitignored):

Variable	Consumed by	Purpose / default
LAVA_API_JACKETT_ENABLED	lava-api-go	turn the Jackett route on (false)
LAVA_API_JACKETT_URL	lava-api-go	http://127.0.0.1:9117 (loopback — NOT a bridge name; see §1)

Variable	Consumed by	Purpose / default
`LAVA_API_JACKETT_APIKEY`	lava-api-go + **healthcheck + validation script**	the Jackett api_key (no default — generated on first run)
`LAVA_API_JACKETT_DEFAULT_INDEXER`	lava-api-go + healthcheck	indexer id for the caps probe (`all`)
`LAVA_JACKETT_IMAGE`	compose	Jackett image — pin by **digest** in real `.env`
`LAVA_FLARESOLVERR_IMAGE`	compose	FlareSolverr image — pin by digest
`LAVA_JACKETT_BIND_HOST` / `LAVA_JACKETT_HOST_PORT`	compose	loopback bind (`127.0.0.1` / `9117`)
`LAVA_FLARESOLVERR_BIND_HOST` / `LAVA_FLARESOLVERR_HOST_PORT`	compose	loopback bind (`127.0.0.1` / `8191`)
`LAVA_JACKETT_CONFIG_DIR`	compose	gitignored `/config` host volume (`./.jackett-config`) — **§6.H secrets store**
`LAVA_JACKETT_PUID` / `_PGID` / `_TZ`	compose	LinuxServer user/ group/timezone
`LAVA_FLARESOLVERR_LOG_LEVEL`	compose	FlareSolverr verbosity (`info`)

> The api_key is the SAME value for the Go service AND the container healthcheck AND the validation script — there is no parallel name to keep in sync. It is a §6.H secret: server-side only, never committed (`.gitignore` excludes `.jackett-config/`), never sent to the device.

4. One-command end-to-end validation

After the api_key is set in `.env` (see §5), the main agent runs:

```
# Jackett only (RuTracker / RuTor / NNMClub indexers — no
        Cloudflare):
scripts/validate-jackett-sidecar.sh

# Jackett + FlareSolverr (the IPTorrents / Cloudflare path):
scripts/validate-jackett-sidecar.sh --cloudflare
```

The script auto-detects podman (preferred) then docker, brings the sidecar up, waits for the compose healthchecks to go HEALTHY, then independently re-probes from the host (the §6.B/§6.J discipline — not trusting `State==running`):

- **Jackett:** `GET .../results/torznab/api?t=caps&apikey=…` → expects HTTP 200 + parseable Torznab XML, api_key not rejected. (Verbatim body printed.)
- **FlareSolverr** (`--cloudflare`): `POST /v1 {"cmd":"sessions.list"}` → expects HTTP 200 + `"status":"ok"`. (Verbatim body printed.)

Exit `0` = READY, `1` = a probe failed (logs tailed), `2` = config error (no runtime / missing api_key). The stack is torn down on exit unless `--keep`.

---

5. One-time Jackett setup (stateful — partly MANUAL)

Jackett's configuration lives in the gitignored `/config` volume and persists across restarts. These steps cannot be fully automated (the dossier's Open Question Q3 — admin HTTP vs per-`/config` JSON — is unresolved):

5.1 Generate the api_key (required before validation passes)

1. Start Jackett once with the config volume mounted (e.g. the validation script with `--keep`, or `docker compose -f tools/lava-containers/docker-compose.jackett.yml --profile jackett up -d` after the network exists). On first run with an empty `/config`, Jackett writes a fresh api_key into `${LAVA_JACKETT_CONFIG_DIR}/Jackett/ServerConfig.json` under the `APIKey` field (or read it from the WebUI top-right "API Key" field at `http://127.0.0.1:9117`).
2. Copy that value into `.env` as `LAVA_API_JACKETT_APIKEY=…`.
3. Re-run the validation script — the caps probe now authenticates. On a zero-indexer config the caps body is an empty `<caps/>`; the script accepts this as "surface live + api_key valid". A real search needs §5.2.

5.2 Add the IPTorrents indexer (with FlareSolverr) — MANUAL WebUI step

IPTorrents sits behind Cloudflare, so FlareSolverr MUST be running and Jackett MUST be pointed at it:

1. Bring up the `cloudflare` profile so `lava-flaresolverr` is running.
2. In the Jackett WebUI (`http://127.0.0.1:9117`) → top-right **gear** / **Settings**: set **FlareSolverr API URL** to `http://lava-flaresolverr:8191` (bridge DNS name — Jackett reaches FlareSolverr over `lava-net`). Save.
3. **Add indexer** → search "IPTorrents" → **+** → enter the IPTorrents credentials (these are a §6.H secret; they live ONLY in the gitignored `/config` volume, never in tracked source). Save.
4. (Optional) Add RuTracker / RuTor / NNMClub indexers the same way — they do not need FlareSolverr, so the `jackett`-only profile suffices for them.

> Per §6.A: if the indexer-add is ever automated via Jackett's admin HTTP API, that automation MUST gain a contract test asserting the request/flag shape (mirroring `lava-api-go/tests/contract/healthcheck_contract_test.go`) and pin the Jackett version, because the admin API surface is version-sensitive.

---

6. How the Android app discovers + uses it

The app never sees Jackett. The flow:

1. App auto-discovers `lava-api-go` via mDNS `_lava-api._tcp` (existing discovery; unchanged by this work).
2. App calls `lava-api-go`'s Torznab-backed search route over TLS+HMAC.
3. `lava-api-go` (holding the api_key server-side) calls Jackett at `http://127.0.0.1:9117/api/v2.0/indexers/<id>/results/torznab/api`, parses the Torznab XML via `internal/jackett`, and returns clean results to the app.
4. Downloads: Jackett answers `/dl/` links with HTTP 302 → `magnet:`; the Go client captures the `Location` verbatim (does NOT auto-follow) and returns a real magnet or `.torrent` bytes (see the implementation doc).

Mapping to `IPTorrentsConfig`

The IPTorrents provider plugin (`core/tracker/iptorrents/.../IPTorrentsConfig.kt`) resolves the **lava-api-go sidecar** base URL (NOT Jackett directly — the app talks only to lava-api-go), in priority order, from:

1. an explicit override,
2. the `iptorrentsJackettBaseUrl` JVM system property (set by the §6.G test task),
3. the `IPTORRENTS_JACKETT_BASE_URL` environment variable (the runtime value).

There is **no compile-time default** (§6.R): unconfigured → `resolve()` returns null → the feature fails honestly ("sidecar base URL not configured") rather than guessing a host. On Android the production value is injected via BuildConfig / runtime config in `:core:tracker:client`. So the chain is:

```
IPTORRENTS_JACKETT_BASE_URL  →  IPTorrentsConfig.resolve()  →
lava-api-go (Torznab route)

→  Jackett (LAVA_API_JACKETT_URL)

→  IPTorrents (via FlareSolverr)
```

---

7. What CANNOT be automated (honest manual list)

Step	Why it's manual	Mitigation
api_key into `.env` (§5.1)	Jackett mints the key on first run into its stateful `/config`; there is no pre-seeding	validation script refuses the placeholder and prints the exact remediation
IPTorrents indexer add + credentials (§5.2)	per-indexer config is WebUI/API state in `/config`; IPTorrents creds are §6.H secrets that must not enter tracked source	documented WebUI steps; future §6.A-contract-tested automation is owed (dossier Q3)
FlareSolverr URL setting in Jackett (§5.2)	a Jackett WebUI/config-file setting, not an env var Jackett reads	one-time WebUI step; persisted in `/config`
Image digest pinning	the real digest is environment-specific; `.env.example` carries the `:latest` placeholder	pin by digest in the real `.env` (§6.K) before any gate run

Everything else — bring-up, healthcheck wait, caps + FlareSolverr probing, teardown — is fully automated by `scripts/validate-jackett-sidecar.sh`.